

RV32I CPU Core - Technical Documentation

Version: 1.0
Date: January 3, 2026
Architecture: RISC-V RV32I Base Integer Instruction Set
Author: Clean-slate implementation for AXIUART project

Table of Contents

- 1. [Overview](#)
 - 2. [Architecture Specifications](#)
 - 3. [Module Interface](#)
 - 4. [Pipeline Architecture](#)
 - 5. [Memory Organization](#)
 - 6. [Instruction Set Summary](#)
 - 7. [Debug Interface](#)
 - 8. [MMIO Support](#)
 - 9. [Hazard Handling](#)
 - 10. [Trace Buffer](#)
 - 11. [Synthesis & Timing](#)
 - 12. [Verification](#)
-

Overview

The RV32I CPU core is a complete implementation of the RISC-V Base Integer Instruction Set (RV32I Version 2.1). It is designed for FPGA deployment with a focus on clean pipeline design, full hazard detection/forwarding, and comprehensive debug capabilities.

Key Features

- **ISA:** Full RV32I compliance (40 instructions)
 - **Pipeline:** Classic 5-stage (IF/ID/EX/MEM/WB)
 - **Registers:** 32 × 32-bit general-purpose (x0 hardwired to zero)
 - **Memory:** 8KB internal block RAM (byte-addressable)
 - **Clocking:** Single-clock domain, synchronous design
 - **Reset:** Active-low asynchronous reset
 - **Hazards:** Full data forwarding, load-use stalls, branch flush
 - **Debug:** External memory access, halt/run/step control
 - **MMIO:** Memory-mapped LED output (expandable)
 - **Trace:** Instruction trace buffer for verification
-

Architecture Specifications

Core Parameters

Parameter	Value	Description
Data Width	32 bits	Register and data bus width
Address Width	32 bits	Byte-addressed memory space
Instruction Width	32 bits	Fixed-width RISC-V instructions
Register Count	32	x0-x31 (x0 = constant 0)
Internal RAM	8 KB	2048 words × 32 bits
Pipeline Stages	5	IF/ID/EX/MEM/WB
Branch Policy	Stall + Flush	No branch prediction
Forwarding	Full	EX→EX, MEM→EX, WB→EX

Physical Characteristics

- **FPGA Resources:** ~1500 LUTs, ~1000 FFs, 1 BRAM block (estimate)
- **Clock Frequency:** Tested at 125 MHz (conservative, can be higher)
- **Critical Path:** ALU → Branch logic → PC mux
- **Latency:** 5 cycles minimum (branch taken), 1 cycle typical (sequential)

Module Interface

Port Definitions

```
module rv32i_core
    import rv32i_isa_pkg::*;
(
    // Clock and Reset
    input logic      clk,           // System clock (125 MHz nominal)
    input logic      rst_n,        // Active-low async reset

    // Debug Control Interface
    input logic      cpu_run,       // Start/resume execution
    input logic      cpu_halt,      // Halt execution
    input logic      cpu_step,     // Single-step (future use)
    output logic     cpu_halted,    // CPU halted status
    output logic     cpu_break,    // Breakpoint hit (EBREAK)

    // Debug Memory Interface (Port B)
    input logic [10:0] dbg_mem_addr, // Word address (11 bits = 2048 words)
    input logic [31:0] dbg_mem_wdata, // Write data
    output logic [31:0] dbg_mem_rdata, // Read data (registered)
    input logic [3:0]  dbg_mem_we,   // Byte write enables [3:0]
    input logic        dbg_mem_re,   // Read enable

    // MMIO Interface
    output logic [3:0] led_out,      // LED output register
```

```
// Trace Buffer Interface
output logic      trace_valid,    // Instruction committed
output logic [31:0] trace_pc,     // Committed PC
output logic [31:0] trace_insn,   // Committed instruction
output logic [4:0]  trace_rd_addr, // Destination register address
output logic [31:0] trace_rd_data // Written register value
);
```

Signal Descriptions

Debug Control

- **cpu_run**: Start or resume CPU execution from halted state. Clears breakpoint flag.
- **cpu_halt**: Request CPU to halt. Takes effect after current instruction completes.
- **cpu_step**: Reserved for single-step execution (not yet implemented).
- **cpu_halted**: Status output indicating CPU is halted (safe for debug access).
- **cpu_break**: Latched signal indicating EBREAK instruction was executed. Cleared by **cpu_run**.

Debug Memory Access

The debug interface provides external read/write access to the CPU's internal RAM when the CPU is halted. This enables:

- Program loading before execution
- Memory inspection during debugging
- Direct testbench access (bypassing UART protocol)

Access Rules:

- Port B has priority over CPU Port A when both access same location
- Debug writes are byte-granular using **dbg_mem_we[3:0]**
- Read latency: 1 clock cycle (registered output)
- Word address range: 0x000 to 0x7FF (2048 words)

Byte Address Mapping:

```
Word Address [10:0] = Byte Address [12:2]
dbg_mem_we[0] = byte 0 (bits 7:0)
dbg_mem_we[1] = byte 1 (bits 15:8)
dbg_mem_we[2] = byte 2 (bits 23:16)
dbg_mem_we[3] = byte 3 (bits 31:24)
```

Pipeline Architecture

Stage Overview

```
IF (Instruction Fetch)
  ↓
ID (Instruction Decode)
  ↓
EX (Execute)
  ↓
MEM (Memory Access)
  ↓
WB (Write Back)
```

Stage Descriptions

IF (Instruction Fetch)

Purpose: Fetch instruction from memory and prepare next PC.

Operations:

- Calculate instruction address: $\text{ram_addr} = \text{PC}[12:2]$ (word address)
- Read instruction from RAM (1-cycle registered read)
- PC increment logic: $\text{PC_next} = \text{PC} + 4$ (sequential)
- Branch target selection (when branch taken)

Outputs:

- insn_if : Fetched instruction (32 bits)
- pc_if : Current program counter

Stall Conditions:

- Load-use hazard detected
- Debug memory access in progress

Flush Conditions:

- Branch/jump taken (wrong-path instruction)
-

ID (Instruction Decode)

Purpose: Decode instruction fields and read register file.

Operations:

- Extract instruction fields (opcode, funct3, funct7, rd, rs1, rs2)
- Generate immediate values (sign-extended for I/S/B/U/J types)
- Read register file (combinational read):
 - $\text{rf_rdata1} = \text{regfile}[\text{rs1}]$ (or 0 if $\text{rs1}=\text{x0}$)
 - $\text{rf_rdata2} = \text{regfile}[\text{rs2}]$ (or 0 if $\text{rs2}=\text{x0}$)
- Generate control signals (ALU op, mem read/write, branch enable)

Outputs:

- `insn_id`: Instruction for next stage
- `pc_id`: PC value for branch calculation
- `rs1_data`, `rs2_data`: Register values
- Control signals: `alu_op`, `mem_write`, `mem_read`, `branch`, `jump`

Hazard Detection:

- Check for load-use hazard:
 - If EX stage is a load (`mem_read=1`)
 - And current instruction uses EX destination register
 - Assert `hazard_load_use` → stall pipeline
-

EX (Execute)

Purpose: Perform ALU operations, calculate addresses, evaluate branches.

Operations:

- **ALU Execution:**
 - Select operands: `rs1` (forwarded), immediate or `rs2` (forwarded)
 - Perform operation: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
- **Branch Evaluation:**
 - Compare `rs1` vs `rs2`: BEQ, BNE, BLT, BGE, BLTU, BGEU
 - Calculate branch target: `PC + sign_extend(imm)`
- **Address Calculation:**
 - Load/Store: `rs1 + sign_extend(imm)`
 - AUIPC: `PC + (imm << 12)`
 - JAL/JALR: Calculate jump target

Outputs:

- `alu_result`: Result for writeback
- `branch_taken`: Branch decision
- `branch_target`: Target PC (if branch taken)
- `mem_addr`: Memory access address

Forwarding Logic:

- Forward from MEM stage if: `(MEM.rd == EX.rs1/rs2) && MEM.rd != 0 && MEM.reg_write`
 - Forward from WB stage if: `(WB.rd == EX.rs1/rs2) && WB.rd != 0 && WB.reg_write`
 - Priority: EX → MEM → WB → RF
-

MEM (Memory Access)

Purpose: Perform load/store operations, MMIO access.

Operations:

- **Load Operations:**
 - Access RAM at calculated address
 - Sign/zero extend based on type: LB, LH, LW, LBU, LHU
 - Support unaligned access (byte/halfword)
- **Store Operations:**
 - Write data to RAM with byte enables
 - SB: Write 1 byte, SH: Write 2 bytes, SW: Write 4 bytes
- **MMIO Detection:**
 - Check if address is in MMIO range (e.g., LED at 0x407C)
 - Route to MMIO registers instead of RAM

Memory Map:

- 0x0000_0000 - 0x0000_1FFF: Internal RAM (8KB)
- 0x0000_4000 - 0x0000_4FFF: MMIO region (4KB)
 - 0x407C: LED output register (4 bits)
- Other addresses: Reserved (no-op)

Outputs:

- `mem_load_data`: Data read from memory/MMIO
- Updated LED register (if MMIO write)

WB (Write Back)

Purpose: Write results back to register file.

Operations:

- Select writeback source:
 - ALU result (R-type, I-type arithmetic)
 - Memory data (Load instructions)
 - PC+4 (JAL, JALR for return address)
 - Upper immediate (LUI, AUIPC)
- Write to register file if: `reg_write && rd != x0`

Outputs:

- `wb_result`: Final result for register file
- `rf_waddr`: Destination register address
- `rf_wen`: Write enable signal

Register File Write:

```
if (rf_wen && rf_waddr != 5'b0) begin
    regfile[rf_waddr] <= rf_wdata;
end
```

Memory Organization

Internal RAM

Configuration:

- **Size:** 8 KB (2048 words × 32 bits)
- **Type:** True dual-port block RAM
- **Addressing:** Byte-addressed (word-aligned preferred)
- **Access:** Single-cycle read latency (registered)

Port Allocation:

Port	Function	Access Pattern
Port A	CPU instruction fetch (IF)	Read-only, word-aligned
Port A	CPU data access (MEM)	Read/write, byte-granular
Port B	Debug/external access	Read/write when CPU halted

Synthesis Attributes:

```
(* ram_style = "block" *)
(* rw_addr_collision = "no" *)
logic [31:0] ram [0:2047];
```

Byte Addressing

RV32I uses byte addressing, but internal RAM is word-organized:

```
Byte Address [31:0] → Word Address [10:0] = Byte Address [12:2]
Byte Offset [1:0] → Determines byte lane selection

Examples:
0x0000_0000 → Word 0, Offset 0 (bytes 3:0)
0x0000_0004 → Word 1, Offset 0 (bytes 7:4)
0x0000_0008 → Word 2, Offset 0 (bytes 11:8)
```

Load/Store Byte Enables

Store Operations:

- SB (Store Byte): 1 byte enable active based on address[1:0]
- SH (Store Halfword): 2 byte enables active
- SW (Store Word): All 4 byte enables active

Example for address 0x0000_0002:

```
SB: byte_en = 4'b0100 (write byte 2 only)
SH: byte_en = 4'b1100 (write bytes 3:2)
SW: byte_en = 4'b1111 (write all bytes)
```

Instruction Set Summary

RV32I Instruction Listing (40 Instructions)

Integer Computational Instructions (19)

Mnemonic	Format	Opcode	Description
ADD	R	0110011	rd = rs1 + rs2
SUB	R	0110011	rd = rs1 - rs2
AND	R	0110011	rd = rs1 & rs2
OR	R	0110011	rd = rs1 rs2
XOR	R	0110011	rd = rs1 ^ rs2
SLL	R	0110011	rd = rs1 << rs2[4:0]
SRL	R	0110011	rd = rs1 >> rs2[4:0] (logical)
SRA	R	0110011	rd = rs1 >> rs2[4:0] (arithmetic)
SLT	R	0110011	rd = (rs1 < rs2) ? 1 : 0 (signed)
SLTU	R	0110011	rd = (rs1 < rs2) ? 1 : 0 (unsigned)
ADDI	I	0010011	rd = rs1 + imm
ANDI	I	0010011	rd = rs1 & imm
ORI	I	0010011	rd = rs1 imm
XORI	I	0010011	rd = rs1 ^ imm
SLLI	I	0010011	rd = rs1 << imm[4:0]
SRLI	I	0010011	rd = rs1 >> imm[4:0] (logical)
SRAI	I	0010011	rd = rs1 >> imm[4:0] (arithmetic)
SLTI	I	0010011	rd = (rs1 < imm) ? 1 : 0 (signed)
SLTIU	I	0010011	rd = (rs1 < imm) ? 1 : 0 (unsigned)

Upper Immediate Instructions (2)

Mnemonic	Format	Opcode	Description
----------	--------	--------	-------------

Mnemonic	Format	Opcode	Description
LUI	U	0110111	rd = imm << 12
AUIPC	U	0010111	rd = PC + (imm << 12)

Load Instructions (5)

Mnemonic	Format	Opcode	Description
LB	I	0000011	rd = sign_extend(mem[rs1+imm][7:0])
LH	I	0000011	rd = sign_extend(mem[rs1+imm][15:0])
LW	I	0000011	rd = mem[rs1+imm]
LBU	I	0000011	rd = zero_extend(mem[rs1+imm][7:0])
LHU	I	0000011	rd = zero_extend(mem[rs1+imm][15:0])

Store Instructions (3)

Mnemonic	Format	Opcode	Description
SB	S	0100011	mem[rs1+imm][7:0] = rs2[7:0]
SH	S	0100011	mem[rs1+imm][15:0] = rs2[15:0]
SW	S	0100011	mem[rs1+imm] = rs2

Branch Instructions (6)

Mnemonic	Format	Opcode	Description
BEQ	B	1100011	if (rs1 == rs2) PC = PC + imm
BNE	B	1100011	if (rs1 != rs2) PC = PC + imm
BLT	B	1100011	if (rs1 < rs2) PC = PC + imm (signed)
BGE	B	1100011	if (rs1 >= rs2) PC = PC + imm (signed)
BLTU	B	1100011	if (rs1 < rs2) PC = PC + imm (unsigned)
BGEU	B	1100011	if (rs1 >= rs2) PC = PC + imm (unsigned)

Jump Instructions (2)

Mnemonic	Format	Opcode	Description
JAL	J	1101111	rd = PC+4; PC = PC + imm
JALR	I	1100111	rd = PC+4; PC = (rs1 + imm) & ~1

System Instructions (3)

Mnemonic	Format	Opcode	Description
ECALL	I	1110011	System call (implementation-defined)
EBREAK	I	1110011	Breakpoint (halts CPU, sets cpu_break)
FENCE	I	0001111	Memory fence (NOP in single-core)

Debug Interface

Debug Control State Machine

The CPU supports three states:

- 1. **HALTED** (cpu_halted=1, running=0)
 - CPU pipeline frozen
 - Safe for external memory access
 - Entry: Power-on reset, cpu_halt assertion, EBREAK instruction
- 2. **RUNNING** (cpu_halted=0, running=1)
 - Normal instruction execution
 - Entry: cpu_run assertion
- 3. **BREAKPOINT** (cpu_break=1)
 - Latched state after EBREAK execution
 - Persists until cleared by cpu_run
 - Allows debugger to detect breakpoint hit

Debug Memory Access Protocol

Write Sequence:

- 1. Assert cpu_halt and wait for cpu_halted=1
- 2. Set dbg_mem_addr (word address)
- 3. Set dbg_mem_wdata (32-bit data)
- 4. Set dbg_mem_we[3:0] (byte enables)
- 5. Wait 1 clock cycle (write completes)

Read Sequence:

- 1. Assert cpu_halt and wait for cpu_halted=1
- 2. Set dbg_mem_addr (word address)
- 3. Assert dbg_mem_re=1
- 4. Wait 1 clock cycle
- 5. Read dbg_mem_rdata (registered output)

Program Loading Example

```

// Halt CPU
cpu_halt = 1;
@(posedge cpu_halted);

// Load program to RAM
for (int i = 0; i < program_size; i++) begin
    @(posedge clk);
    dbg_mem_addr = i;
    dbg_mem_wdata = program[i];
    dbg_mem_we    = 4'b1111; // Write all 4 bytes
end

// Start execution
@(posedge clk);
cpu_halt = 0;
cpu_run = 1;

```

MMIO Support

LED Output Register

Address: `0x0000_407C` (byte address)

Width: 4 bits

Access: Write-only

Reset Value: `4'h0`

Usage:

```

LUI   x5, 0x00004    # x5 = 0x00004000
ADDI  x6, x0, 0x5     # x6 = 5 (LED pattern)
SW    x6, 0x7C(x5)    # Write to LED register

```

Hardware Implementation:

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        led_reg <= 4'h0;
    end else if (mem_write && mem_addr == 32'h0000_407C) begin
        led_reg <= mem_wdata[3:0];
    end
end
end

```

Adding New MMIO Registers

To add a new MMIO device:

1. Define address in MMIO range (0x4000-0x4FFF)
2. Add register in MEM stage:

```
logic [31:0] my_device_reg;
```

3. Add write logic:

```
else if (mem_write && mem_addr == MY_DEVICE_ADDR) begin
    my_device_reg <= mem_wdata;
end
```

4. Add read mux:

```
case (mem_addr)
    LED_ADDR: mem_load_data = {28'h0, led_reg};
    MY_DEVICE_ADDR: mem_load_data = my_device_reg;
    default: mem_load_data = ram_rdata_mem;
endcase
```

Hazard Handling

Data Hazards

Types:

1. **RAW (Read After Write)** - Most common
2. **WAW (Write After Write)** - Not possible in-order pipeline
3. **WAR (Write After Read)** - Not possible with register read in ID

Forwarding Logic

Forwarding Paths:

```
EX/MEM → EX (forward from MEM stage to EX ALU inputs)
MEM/WB → EX (forward from WB stage to EX ALU inputs)
```

Forwarding Conditions:

For rs1:

```
if (EX_MEM.reg_write && EX_MEM.rd != 0 && EX_MEM.rd == ID_EX.rs1)
    forward_rs1 = 2'b01; // Forward from MEM
```

```

else if (MEM_WB.reg_write && MEM_WB.rd != 0 && MEM_WB.rd == ID_EX.rs1)
    forward_rs1 = 2'b10; // Forward from WB
else
    forward_rs1 = 2'b00; // Use RF value

```

Similar logic for rs2.

Load-Use Hazard

Problem: Load instruction in EX stage, dependent instruction in ID stage.

Example:

```

LW  x5, 0(x10)    # EX: Load from memory (result not ready)
ADD x6, x5, x7     # ID: Needs x5 value (hazard!)

```

Solution: Stall ID stage for 1 cycle.

Detection:

```

hazard_load_use = (ID_EX.mem_read) &&
                  ((ID_EX.rd == rs1) || (ID_EX.rd == rs2)) &&
                  (ID_EX.rd != 0);

```

Effect:

- Insert 1-cycle bubble in pipeline
- Freeze IF and ID stages
- Allow EX stage to complete (data ready in MEM)
- Resume with forwarding from MEM stage

Control Hazards (Branches)

Policy: Predict not-taken, flush on mispredict.

Mechanism:

1. Branch evaluated in EX stage
2. If taken: Flush IF, ID stages (wrong-path instructions)
3. Update PC to branch target
4. Resume fetch from target

Penalty: 2 cycles on taken branch/jump

Trace Buffer

Purpose

The trace buffer captures executed instructions for verification and debugging without UART overhead. UVM testbenches can directly access the trace buffer via SystemVerilog handles.

Trace Entry Format (128 bits)

[127:96]	PC (32 bits)	- Program counter
[95:64]	Instruction (32 bits)	- Executed instruction
[63:32]	rd_value (32 bits)	- Result written to rd
[31:27]	rd_addr (5 bits)	- Destination register address
[26:0]	Reserved (27 bits)	- Reserved for future use

Trace Buffer Module

File: rv32i_trace_buffer.sv
Depth: 64 entries (configurable)
Access: Direct UVM memory access (no protocol)

Usage in Testbench

```
// In testbench:
Rv32i_Trace_Buffer trace_buf (
    .clk(clk),
    .rst_n(rst_n),
    .insn_valid(trace_valid),
    .insn(trace_insn),
    .pc(trace_pc),
    .rd_addr(trace_rd_addr),
    .rd_value(trace_rd_data),
    .trace_buffer(trace_buffer_export),
    .write_ptr(trace_wr_ptr),
    .entry_count(trace_count)
);

// In UVM monitor:
for (int i = 0; i < trace_count; i++) begin
    trace_entry_t entry = trace_buffer_export[i];
    `uvm_info("TRACE", $sformatf("PC=%08h INSN=%08h RD=x%0d VAL=%08h",
        entry.pc, entry.insn, entry.rd_addr, entry.rd_value), UVM_LOW)
end
```

Synthesis & Timing

Resource Utilization (Xilinx 7-Series)

Resource	Usage (est.)	Percentage
LUTs	~1500	<1% (XC7Z020)

Resource	Usage (est.)	Percentage
Flip-Flops	~1000	<1% (XC7Z020)
Block RAM	1 (8KB)	<1% (XC7Z020)
DSP Slices	0	0%

Timing Constraints

Target Clock: 125 MHz (8 ns period)

Critical Paths:

- 1. ALU → Branch comparator → PC mux (~6 ns)
- 2. Register file read → ALU → Forwarding mux (~5.5 ns)
- 3. Memory read → Sign extend → WB mux (~4 ns)

Recommended Constraints:

```
create_clock -period 8.0 [get_ports clk]
set_input_delay -clock clk 2.0 [all_inputs]
set_output_delay -clock clk 2.0 [all_outputs]
```

Optimization Guidelines

- 1. **Pipeline Balancing:** EX stage is critical path (consider ALU register slicing)
- 2. **RAM Inference:** Use `(* ram_style = "block" *)` to force BRAM usage
- 3. **Retiming:** Enable register balancing in synthesis for better Fmax
- 4. **Forwarding Mux:** Wide multiplexers can be critical (consider early evaluation)

Verification

Test Strategy

- 1. **Instruction-Level Tests:** Each RV32I instruction verified individually
- 2. **Hazard Tests:** Forwarding, load-use, branch flush scenarios
- 3. **Memory Tests:** Load/store with various alignments and sizes
- 4. **MMIO Tests:** LED register write verification
- 5. **Debug Tests:** External memory access, halt/run control
- 6. **Regression Suite:** Full ISA compliance verification

UVM Testbench Structure

Location: `sim/tests/rv32i_*.sv`

Key Tests:

- `rv32i_basic_test.sv`: Smoke test (26 instructions, LED verification)

- `rv32i_debug_load_test.sv`: Debug interface proof-of-concept
- (Add more as needed)

Verification Checklist

- ☐ All 40 RV32I instructions execute correctly
- ☐ x0 hardwired to zero (write ignored, always reads 0)
- ☐ Data forwarding paths validated
- ☐ Load-use hazard stall works
- ☐ Branch/jump flush behavior correct
- ☐ Byte/halfword load with sign/zero extension
- ☐ Byte-granular store operations
- ☐ EBREAK halts CPU and sets `cpu_break`
- ☐ Debug memory access with CPU halted
- ☐ LED MMIO register updates correctly
- ☐ Trace buffer captures all executed instructions

Appendix: Register File Conventions

ABI Register Names

Register	ABI Name	Saver	Description
x0	zero	-	Hardwired zero
x1	ra	Caller	Return address
x2	sp	Callee	Stack pointer
x3	gp	-	Global pointer
x4	tp	-	Thread pointer
x5-x7	t0-t2	Caller	Temporaries
x8	s0/fp	Callee	Saved / Frame pointer
x9	s1	Callee	Saved register
x10-x11	a0-a1	Caller	Function arguments / return values
x12-x17	a2-a7	Caller	Function arguments
x18-x27	s2-s11	Callee	Saved registers
x28-x31	t3-t6	Caller	Temporaries

Revision History

Version	Date	Author	Changes
1.0	2026-01-03	GitHub Copilot	Initial documentation for RV32I migration

References

1. **RISC-V Specification:** <https://riscv.org/technical/specifications/>
 2. **RV32I ISA Volume 1:** Unprivileged Specification Version 20191213
 3. **Patterson & Hennessy:** Computer Organization and Design RISC-V Edition
 4. **Project Repository:** MameMame777/TD4UART (feature/cpu-mmio-led branch)
-

End of Documentation